HolmesGPT

Enquêter sur un incident
Agir sous contrôle

L'observabilité au service du LLM

Cédric DELGEHIER

01

Pourquoi HolmesGPT ?

Une alerte signale un symptôme. Pour décider de la suite, il faut encore choisir les vérifications, lire leurs résultats et expliquer ce qui se passe.

Comment passer de cette alerte à une investigation compréhensible, puis à une réparation maîtrisée ?

HolmesGPT, un agent d'investigation open source

Créé à l'origine par **Robusta.dev**, HolmesGPT a rejoint la **Cloud Native Computing Foundation** au niveau Sandbox le **8 octobre 2025**.

Il aide à diagnostiquer les incidents en consultant les outils déjà disponibles : métriques, logs, traces, état des services et historique des changements. Son périmètre peut couvrir Kubernetes, des VM, des services cloud ou des bases de données.

Le moteur d'enquête et le modèle de langage sont deux composants distincts. Le fournisseur du LLM est configurable ; le modèle choisit les vérifications et interprète les réponses des outils auxquels Holmes a accès.

L'objectif est de réduire le travail d'investigation et de préparer une réponse étayée. La durée et la qualité du diagnostic restent à mesurer sur les incidents réels.

Ce qui déclenche une enquête

Une enquête peut partir d'une alerte, d'une question humaine, d'un déploiement ou d'une vérification planifiée.

Déclencheur	Exemple d'usage
Après une alerte	Un relais reçoit l'alerte et transmet le contexte de l'incident à Holmes.
À la demande	Un opérateur pose une question via la CLI ou une interface reliée à Holmes.
Après un déploiement	Comparer erreurs, latence et logs avant et après le changement.
Selon un calendrier	Rechercher régulièrement une dégradation sur un service donné.

Dans la VM de démonstration, Alertmanager déclenche les enquêtes via notre relais HTTP.

Ce que Holmes ajoute à l'alerte

Le point de départ

Une alerte « service HTTP indisponible » indique qu'un contrôle échoue. Elle n'explique pas encore si le processus est arrêté, si une dépendance répond mal ou si un fichier est devenu illisible.

Sans enquête automatisée, une personne doit choisir les sources, lancer les vérifications, lire leurs résultats et décider de la suite.

Le travail attendu de Holmes

Holmes consulte les métriques, l'état et les journaux accessibles, puis confronte ces observations aux hypothèses.

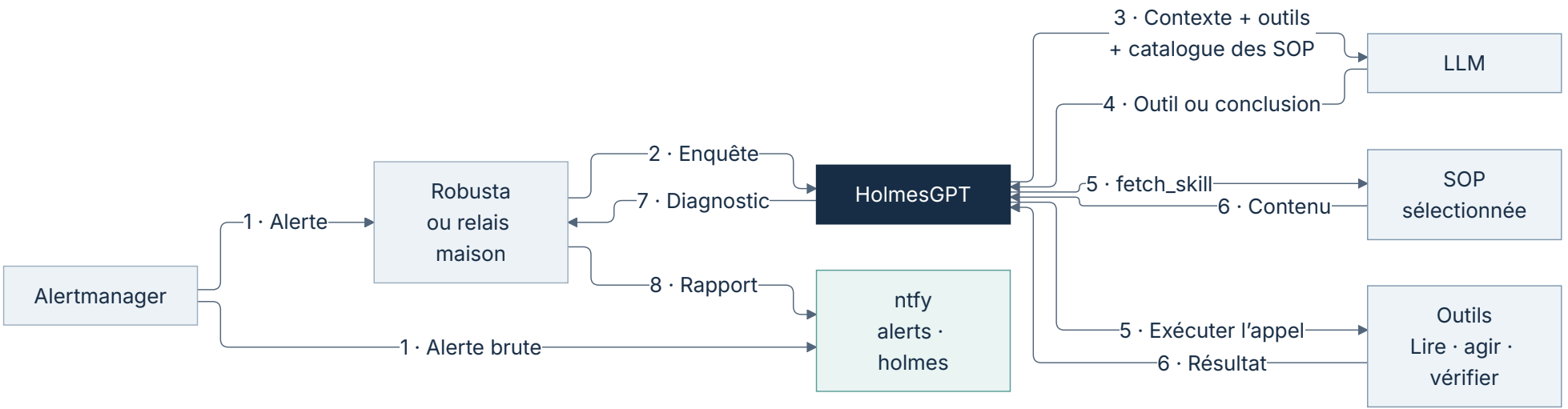
Si une **Standard Operating Procedure (SOP)** existe, elle guide les vérifications et les conditions d'action. Sinon, le modèle détermine les vérifications à partir de l'alerte et des résultats des outils disponibles.

Son rapport doit réduire le travail restant pour l'humain. Une réparation n'est possible que si un outil existe et si ses contrôles l'autorisent.

La valeur ajoutée se juge sur l'enquête produite et la suite proposée, pas sur la reformulation de l'alerte.

Après une alerte

Holmes envoie le contexte et les catalogues au LLM. Le modèle demande un outil - dont `fetch_skill` pour une SOP - puis reçoit son résultat pour décider de la suite.



La boucle 3–6 se répète : la SOP chargée et les résultats enrichissent le contexte envoyé au LLM. Il peut demander un nouvel outil ou conclure, dans la limite de `MAX_STEPS`.

Dans le POC : relais maison ; topic `alerts` pour l'alerte brute, `holmes` pour le rapport.

Les décisions du modèle et les limites des outils

Les pannes connues de cette démo pourraient être traitées par un script.
Le POC sert à observer comment Holmes exploite les preuves et prépare la suite de l'incident.

Le modèle (LLM) conduit l'enquête

Il propose des hypothèses et demande des lectures avec les outils disponibles. La SOP lui indique les vérifications attendues et les branches à suivre.

Il interprète les réponses, demande une action permise ou explique pourquoi il faut passer la main. Son rapport doit citer les observations qui justifient sa conclusion.

Les outils et leurs permissions imposent les limites

Les permissions définissent les ressources accessibles et les actions possibles. **Les contrôles des outils peuvent refuser une demande du modèle.**

Dans cette démo, notre relais Python (substitut de Robusta) vérifie l'incident, les lectures requises et la cible avant d'agir, puis contrôle le résultat.

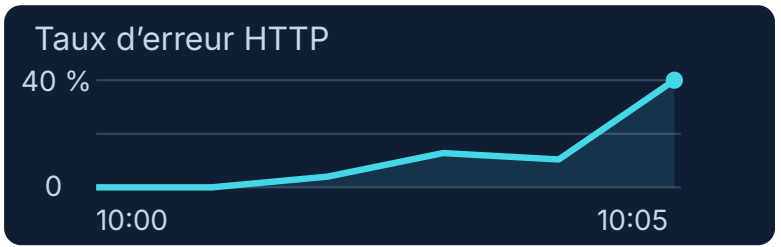
02

Comprendre le moteur d’investigation

Holmes relie un modèle de langage à des outils. Le modèle choisit les vérifications ; les outils donnent accès aux observations réelles.

Après le parcours de l’alerte, regardons les preuves que les outils peuvent consulter.

Ce que chaque source permet d'observer

Source	Ce qu'on observe	Exemple illustratif
Métriques Prometheus Des valeurs suivies dans le temps		Le taux d'erreur passe de **0 à 40 %**. On voit quand le service se dégrade et l'ampleur de l'impact.
Logs Loki Des événements datés		Le journal signale un **refus d'accès au fichier**. On obtient le message d'erreur et son horodatage.
Traces Jaeger · OTel Les opérations d'une requête		La requête échoue dans **la lecture du fichier**. On relie l'erreur à son opération, sa durée et sa requête parente.

État courant : systemd (service actif ?), montages et permissions Linux ; API Kubernetes (statut des pods).

03

Construire notre environnement

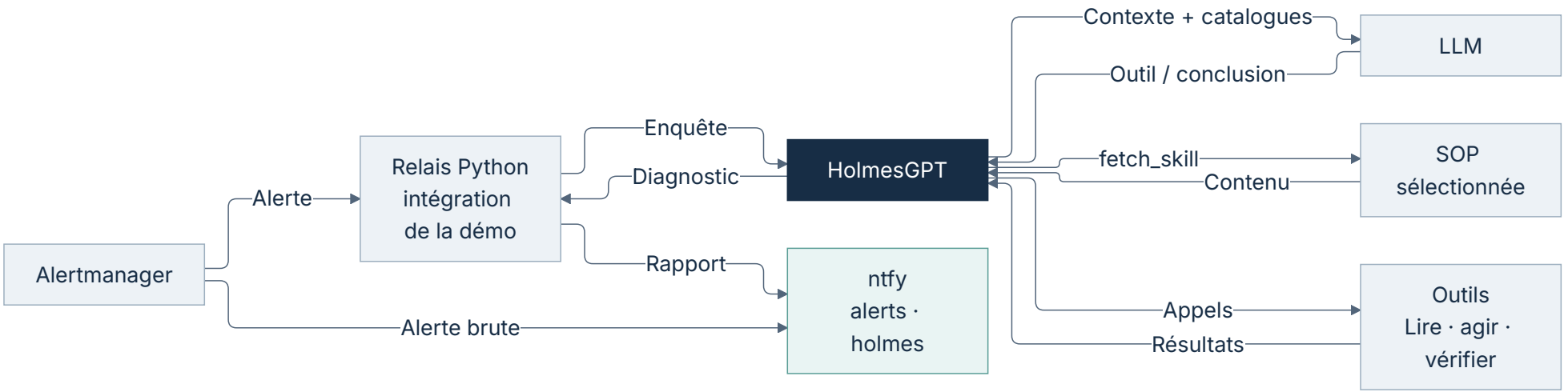
Une VM Linux héberge un service HTTP, sa collecte d’observabilité et le relais vers Holmes. Le serveur du modèle est accessible par API.

Le parcours de la démo : application → observations → alerte → enquête → rapport.

La VM relie l'application, les preuves et Holmes

FastAPI sert la page et `/api/check`, qui lit le fichier de démonstration.

Observabilité : Prometheus, Alertmanager, Blackbox Exporter, Node Exporter, OpenTelemetry Collector, Jaeger, Loki et Grafana.



Les outils interrogent Prometheus, Loki, Jaeger et l'état Linux. Grafana permet à l'humain d'explorer ces signaux. Le diagnostic revient au relais, qui le publie dans ntfy.

Logs et traces : deux lectures d'une requête

Le log donne le détail de l'événement

L'API FastAPI écrit un événement daté avec son message, son niveau et ses champs. Ici, il explique pourquoi la lecture a échoué.

```
{
  "message": "Demo file cannot be read",
  "error_type": "PermissionError",
  "trace_id": "…"
}
```

Ce message peut être très précis, mais ne décrit pas automatiquement toutes les opérations de la requête ni leurs relations.

La trace donne le parcours de la requête

Une trace regroupe des **spans**. Un span représente une opération avec un début, une fin, un statut et éventuellement un parent.



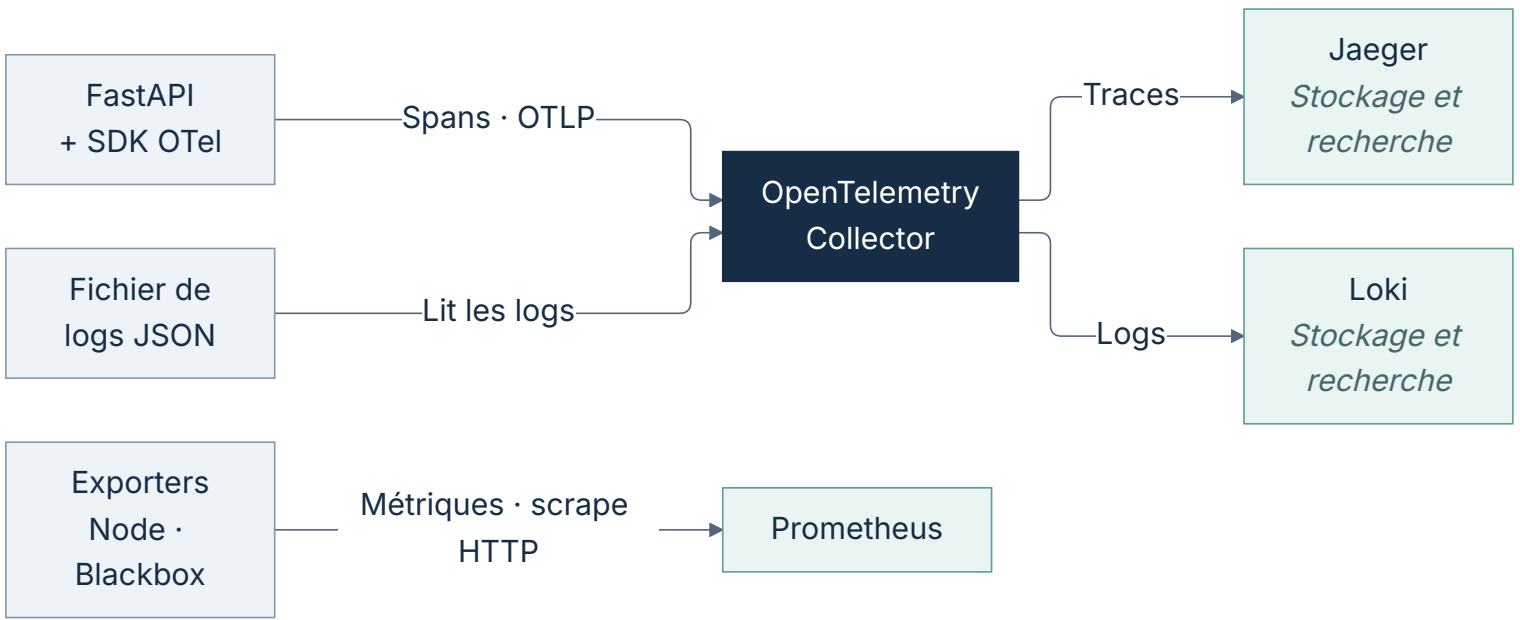
La barre HTTP représente le **span parent** ; la lecture du fichier est son **span enfant**. La trace situe l'opération en erreur dans la requête.

Un même `trace_id` permet de passer du log détaillé à la trace de la requête.

Des signaux aux backends d'observabilité

```python
with tracer.start_as_current_span("read demo message"):
    message = message_file.read()
```

L'instrumentation FastAPI crée le span HTTP. Ce code ajoute un **span enfant** ; le SDK exporte les spans en arrière-plan.



Le Collector lit les logs JSON et le journal systemd. **Prometheus interroge les exporters :** Node Exporter mesure l'hôte, Blackbox Exporter sonde FastAPI.

04

Faire vivre une enquête

La sonde HTTP échoue alors que l’API FastAPI tourne encore. Nous allons suivre une erreur de lecture du fichier, depuis le symptôme jusqu’au résultat de l’intervention.

Quelles preuves permettent d’autoriser une réparation, puis d’affirmer que le service est rétabli ?

La sonde échoue, mais le processus tourne

Dans la démo, l'API FastAPI lit `/srv/demo/message.txt` pour répondre. Nous allons retirer ses permissions de lecture, sans arrêter le processus.

```
# Sur la VM
sudo chmod 000 /srv/demo/message.txt
```

FastAPI reste actif et sert sa page d'accueil, mais `/api/check` renvoie maintenant HTTP 503. Son **log** indique `PermissionError` et sa **trace** localise l'échec sur la lecture du fichier. Prometheus constate l'échec de la sonde, puis Alertmanager envoie l'alerte au relais.

L'alerte signale une réponse HTTP en échec. Elle ne démontre ni un processus arrêté ni une permission à modifier : l'enquête doit encore vérifier la cause.

Des preuves aux décisions de Holmes

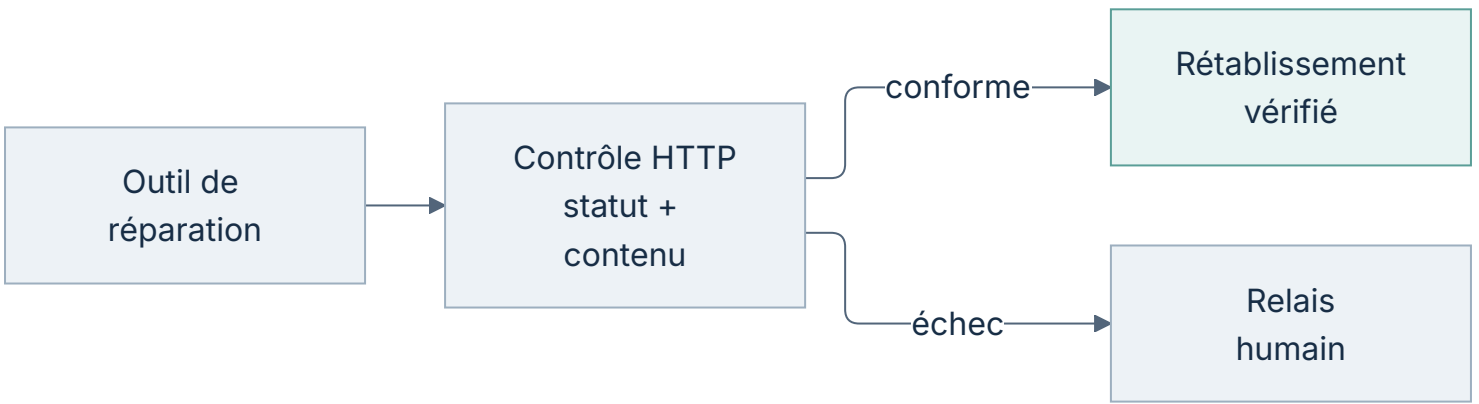
Chaque preuve précise l'hypothèse et guide la prochaine vérification.

Preuve recueillie	Ce que Holmes en déduit	Vérification suivante
Sonde : HTTP `503`	Le service répond, mais le traitement échoue.	Localiser l'échec dans la trace.
Span `read demo message` en erreur	La lecture échoue ; sa cause reste à préciser.	Lire le log associé au même `trace_id`.
Log : `PermissionError`	L'accès au fichier est refusé.	Contrôler le fichier, le montage et l'utilisateur du service.
Fichier présent, mode `000` ; montage et service actifs	Le fichier est illisible par le service.	Vérifier la SOP et l'outil de réparation autorisé.

Conclusion : défaut de permissions. La SOP et l'outil encadrent la réparation.

Une action réussie ne suffit pas

Une commande peut réussir alors que l'incident continue : le processus démarre, mais une dépendance reste inaccessible ou la réponse HTTP n'a pas le contenu attendu. Le succès de l'action doit donc être suivi d'un contrôle du service.



Dans la démo, l'outil de réparation refait une requête et vérifie **HTTP 200 et le contenu attendu**. Si ce contrôle échoue, Holmes doit rapporter « action exécutée, rétablissement non confirmé » plutôt que déclarer l'incident résolu.

05

Maîtriser l’agent

Nous avons vu une réparation possible. Il reste à définir quand elle est permise, quand l’agent doit s’arrêter et quelles ressources son enquête peut consommer.

La SOP guide les décisions ; les outils et leurs permissions imposent les limites.

Dans la SOP : cadrer l'enquête

Une SOP doit permettre à Holmes de reconnaître le bon incident, de choisir les vérifications utiles et de comprendre ce que leurs résultats permettent de conclure.

À préciser	Exemple pour un service HTTP
Déclencheur et périmètre	Nom de l'alerte, service et environnement visés, propriétaire et exceptions telles qu'une maintenance connue.
Comportement attendu	HTTP 200 et contenu attendu : le seul fait que le processus soit actif ne démontre pas la santé du service.
Hypothèses et preuves	Quelles lectures distinguent service arrêté, dépendance en erreur et fichier illisible ? Qu'est-ce qui reste inconnu ?
Outils et collecte	Nommer les outils, la période à consulter et les champs utiles ; borner le volume sans masquer les erreurs.

Exemple : une SOP pour le service HTTP

```
---
name: demo-http-service
description: Diagnostiquer DemoHttpUnavailable.
---
1. Lire l'état, les métriques et les logs.
2. Confirmer : sonde en échec ET service arrêté.
3. Appeler start_demo_fastapi une fois si autorisé.
4. Rapporter le résultat HTTP vérifié.
```

Extrait de la SOP HTTP, abrégé et traduit

Versionner la SOP, puis la rejouer sur une panne et sur un cas où l'action doit être refusée.

Une SOP, un fichier

```
sops/
├── http-service/SKILL.md
├── file-permissions/SKILL.md
├── cpu-pressure/SKILL.md
├── memory-pressure/SKILL.md
├── disk-space/SKILL.md
├── missing-mount/SKILL.md
└── monitoring/SKILL.md
```

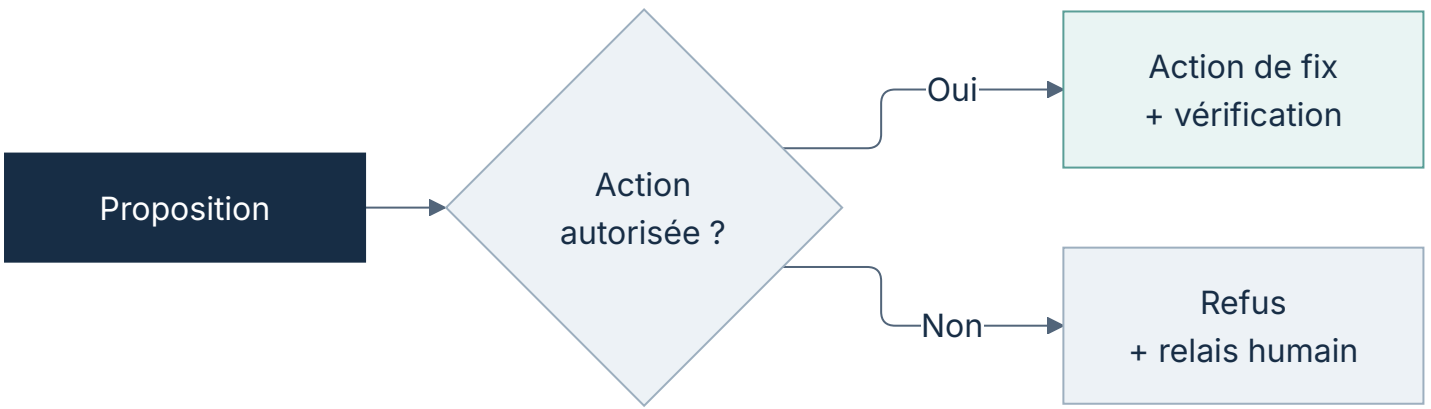
Nom et description guident le choix. Le contenu précise les preuves, les actions permises et l'escalade.

Robusta ou notre relais transmet l'alerte ; le modèle choisit la SOP dans le catalogue de Holmes et demande `fetch_skill`. Le nom et la description guident ce choix.

La liberté d’agir est explicite

La SOP guide l’enquête
Holmes charge la procédure de l’incident : preuves à réunir, conditions de réparation et situations qui nécessitent un humain. La SOP ne donne aucun droit système.

Les outils bornent les actions
Trois actions fixes sont disponibles : démarrer le service HTTP, restaurer les permissions du fichier ou arrêter le stress CPU identifié. L’exécuteur vérifie chaque demande. La RAM reste une intervention humaine.



L’exécuteur exige un incident actif, les preuves attendues et la cible prévue. Il limite les tentatives et vérifie le rétablissement ; un refus ou un échec est transmis à l’humain.

Une SOP cadre les pistes et les refus d’agir

La SOP guide les vérifications, les changements de piste et les refus d’action.

Constat	Décision décrite dans la SOP
API atteinte + `PermissionError`	Vérifier fichier, montage et droits avant DNS ou réseau. Cette priorité repose sur une preuve.
Fichier ou montage absent, ou preuve contradictoire	Sortir de la SOP de permissions et réviser l’hypothèse. Une autre SOP peut devenir pertinente.
Outil indisponible, conditions inconnues ou action refusée	Passer la main avec les preuves et les vérifications manquantes, sans forcer une action ni élargir les droits.

Après une action, vérifier le rétablissement. Un échec exige un rapport et une reprise humaine.

Dans la SOP : agir, vérifier ou passer la main

La procédure doit décrire les conditions de réussite et les cas où l'enquête s'arrête. « Réparer si nécessaire » ne définit ni une autorisation ni une vérification.

À préciser	Décision attendue
Conditions d'action	Préciser les preuves à réunir, la cible exacte et l'outil autorisé avant toute modification.
Limites et repli	Fixer les tentatives et la conduite à tenir après un refus ou un échec ; aucun élargissement improvisé des droits.
Rétablissement	Refaire le contrôle de santé après l'action : statut HTTP et contenu attendu, plutôt que le seul succès de la commande.
Arrêt et rapport	En cas de doute ou d'absence de progrès, transmettre faits, actions, inconnues et prochaine vérification à un responsable identifié.

06

Pourquoi une enquête échoue

Le modèle dépend des lectures qu’il reçoit et des ressources disponibles. Une enquête peut s’arrêter parce que ses outils échouent, que le contexte devient trop volumineux ou que la mémoire manque.

Nous partons des échecs observés pour comprendre les budgets d’étapes, de tokens et de mémoire.

Quand l’enquête s’allonge sans conclure

Une enquête incomplète peut avoir plusieurs causes. Avant de changer le modèle ou d’augmenter une limite, il faut retrouver le premier échec dans les appels et leurs résultats.

Symptôme	Piste à vérifier	Lecture utile
Tours épuisés	Question trop large ou lectures répétées sans progrès.	Chronologie des appels et hypothèses encore ouvertes.
Contexte trop volumineux	Logs trop longs, historique encombré ou compactions répétées.	Taille des entrées, filtres et résumés envoyés au modèle.

Des lectures répétées peuvent venir d’une hypothèse mal délimitée, d’un résultat inutilisable ou d’une information perdue dans l’historique. La chronologie permet de choisir ce qu’il faut corriger.

Le plafond de tours et la taille du contexte répondent à deux problèmes différents. Leurs réglages doivent préserver les preuves nécessaires au diagnostic.

Pourquoi limiter les tours d'enquête

Un plafond configurable

`MAX_STEPS` borne les tours d'enquête ; un tour peut appeler plusieurs outils. Chaque lecture doit apporter une preuve nouvelle ou vérifier un changement.

8 était la limite des premiers essais. Ce n'est pas une règle universelle de Holmes.

La démo utilise un plafond de **12 tours**. Ce nombre laisse plusieurs vérifications possibles ; ce n'est ni une garantie de diagnostic ni une cible à atteindre.

Ce qu'on a observé

Une investigation trop large épuisait les **8 tours**, sans diagnostic exploitable.

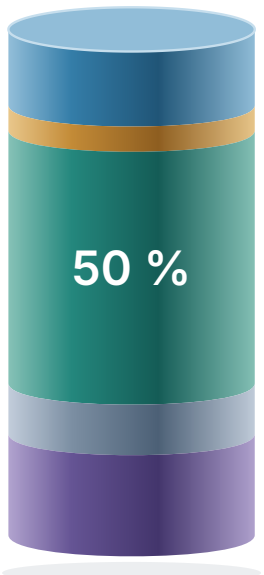
Passer à **16 tours** n'avait pas suffi : certains outils échouaient et renvoyaient trop de texte.

Des questions ciblées et des lectures bornées ont ensuite donné des rapports exploitables. Il fallait rendre les observations utilisables, et pas seulement donner davantage de tours au modèle.

Un budget plus grand ne corrige pas un outil en échec ou une enquête mal délimitée.

Répartir le contexte avant de le remplir

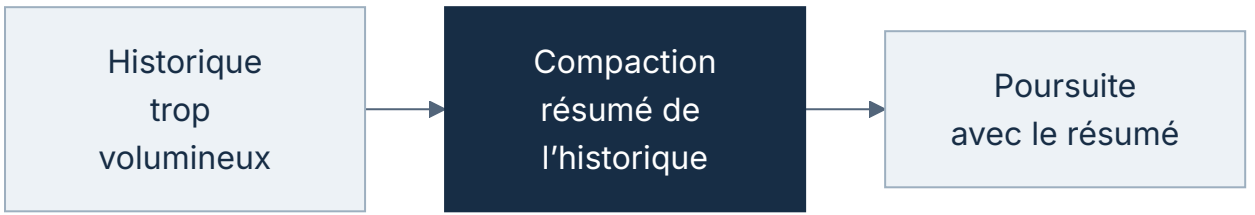
À chaque appel, Holmes renvoie au modèle les consignes, les descriptions d'outils et l'historique utile. Le modèle travaille sur ce contenu ; il ne possède pas une mémoire durable implicite de l'incident.



Consignes, SOP et descriptions d'outils	**3 072** tokens 15 % du contexte
Question et faits de l'incident	**1 024** tokens 5 % du contexte
Historique utile et résultats d'outils	**10 240** tokens 50 % du contexte
Marge pour la variation des entrées	**2 048** tokens 10 % du contexte
Réserve de sortie	**4 096** tokens 20 % du contexte

Budget total : `max_context_size: 20480` . `max_tokens: 4096` par appel, pas pour toute l'enquête. Répartition indicative des entrées.

La compaction : résumer pour continuer



Gain de place
Holmes demande au modèle de résumer l'historique, puis poursuit avec ce résumé. L'enquête peut continuer sans renvoyer toutes les lectures précédentes. Cet appel coûte du temps et des tokens, **sans consommer de tour supplémentaire dans** `MAX_STEPS`.

Perte possible de détails
Le résumé peut omettre une erreur exacte, une date ou une vérification qui avait écarté une hypothèse. Le modèle risque alors de recommencer une lecture ou de revenir à une piste déjà contredite.

Filtrer les résultats aux preuves utiles réduit le besoin de compacter.

Réduire les tokens sans perdre les preuves

Avant l'enquête

Présenter les noms et descriptions des SOP, puis charger le contenu pertinent avec `fetch_skill`. Injecter tous les fichiers complets consommerait du contexte inutilement.

Préciser la ressource, la période et le comportement attendu : le modèle peut alors demander une lecture ciblée, au lieu de parcourir tout le système pour chercher un problème quelconque.

Dans les résultats

Filtrer les logs par service et par période ; limiter les lignes et les champs au niveau de la source. Lire un énorme résultat pour le couper ensuite reste coûteux.

Dédupliquer les répétitions en conservant l'erreur, la date, la cible et l'identifiant de trace. Ces détails permettent de relier les observations et de retrouver les données originales.

Signaler les données absentes ou tronquées pour que le rapport n'en tire pas une fausse conclusion.

Quand les lectures ne parviennent pas au modèle

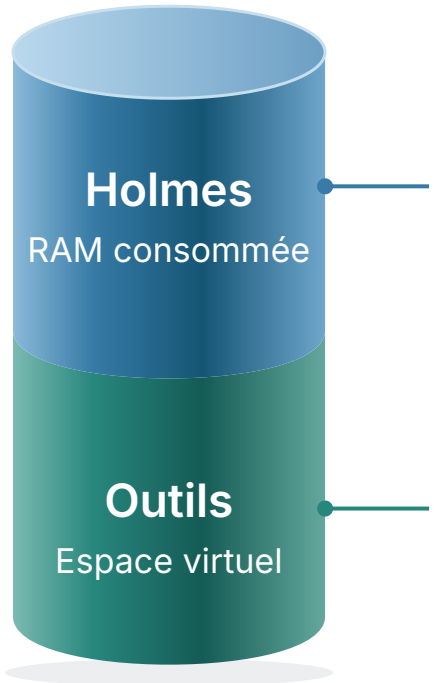
Une lecture peut échouer avant de fournir la moindre observation. Il faut alors examiner l'exécution de l'outil et la disponibilité des composants qui portent l'enquête.

Symptôme	Piste à vérifier	Lecture utile
Outil en échec	Commande qui ne démarre pas, accès refusé ou source indisponible.	Erreur exacte, code de sortie et lecture rejouée seule.
Enquête interrompue	Mémoire du conteneur, serveur LLM ou timeout.	Journaux, événements OOM et consommation par composant.

Une erreur mémoire d'un sous-processus, un OOM du conteneur Holmes et un serveur LLM indisponible demandent des corrections différentes. Un timeout ne suffit pas à identifier lequel a échoué.

Rejouer la lecture seule permet de tester l'outil avant de juger la réponse du modèle.

Ajuster la mémoire dans la bonne couche



Le conteneur Holmes

La limite porte sur la **RAM consommée** par Holmes et ses outils (`mem_limit: 1536m` dans la démo). Un dépassement peut provoquer un **OOM**.

Leviers : filtrer les résultats à la source, borner les enquêtes simultanées, puis dimensionner sur les pics mesurés avec une marge.

Chaque processus d'outil

`TOOL_MEMORY_LIMIT_MB` borne l'**espace d'adressage virtuel** par commande. Le runtime Go de `kubectl` peut réserver beaucoup d'espace au démarrage sans consommer autant de RAM.

Levier : rejouer la lecture seule ; ajuster ce plafond si le refus de réservation est confirmé.

Serveur LLM — RAM/VRAM : mesurer la consommation du modèle et du cache ; ajuster le contexte, la concurrence, la quantification ou la capacité.

Démo : RAM, CPU et fichier absent

01 RAM : diagnostiquer et rapporter

`task fault:memory` — Holmes constate la pression mémoire et envoie un rapport dans ntfy. Il prépare l'intervention humaine : aucun outil ne lui permet d'arrêter le processus consommateur.

02 CPU : identifier, arrêter et vérifier

`task fault:cpu` — Holmes identifie le job de stress et appelle l'outil d'arrêt autorisé. L'outil vérifie la baisse du CPU et la résolution de l'alerte, puis Holmes rapporte le résultat.

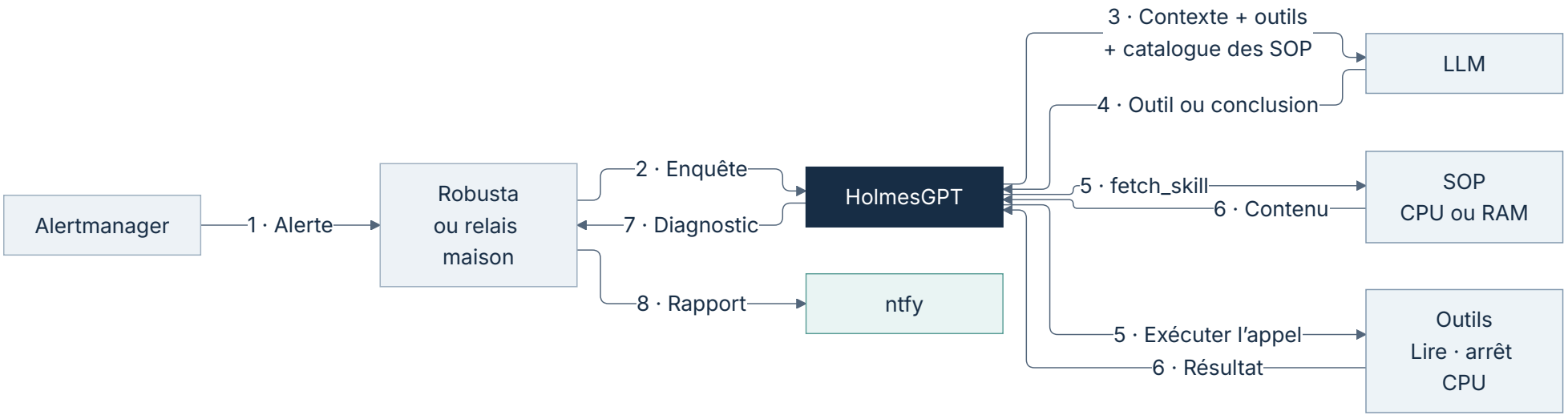
03 Fichier absent : FastAPI tourne, une route échoue

`task fault:file` — FastAPI reste actif et la page d'accueil répond, mais `/api/check` renvoie HTTP 503. Holmes relie logs et traces au fichier manquant et demande une intervention humaine.

Grafana Alertes Rapports Holmes Jaeger

Même enquête, un outil d'action en plus

RAM et CPU : même boucle d'enquête, mêmes lectures ; une SOP adaptée à chaque alerte.



RAM : diagnostic et intervention humaine. **CPU :** `stop_demo_cpu_stress` arrête le job identifié, puis vérifie le CPU sous 85 % et l'alerte résolue.

La SOP guide le diagnostic ; l'outil autorisé permet d'agir et de vérifier.

Retour au récit : vérifier le rétablissement

Ce qu'on reçoit dans ntfy

Notification réelle du 4 novembre · extrait traduit et abrégé

Holmes demo topic `holmes` · 11:36

Holmes — DemoCpuBusy

Faits. CPU hôte à 99,98 %. Le job `labstack-demo-cpu.service` est actif et consomme 98,4 % du CPU. FastAPI répond HTTP 200.

Cause identifiée. Le job `stress-ng` sature le CPU de la VM.

Action. Appel de `stop_demo_cpu_stress` pour arrêter ce job.

> **Outil : rétablissement vérifié.** Job CPU arrêté, CPU hôte sous 85 % et alerte `DemoCpuBusy` résolue.

Incident : `340f34c77e58`